

TUTORIAL PRÁTICO · .NET 8 · WEB API

Repositórios, Use Cases e Autenticação JWT

Passo a passo da implementação das camadas de Usuário e Aluno com geração e validação de tokens JWT no padrão Clean Architecture.

Projeto

Einstein Gestão Acadêmica

Plataforma

.NET 8 · ASP.NET Core

Banco de Dados

PostgreSQL + EF Core 9

Autenticação

JWT Bearer Token

Sumário

FUNDAMENTOS

- 0 Arquitetura do Projeto — As 4 Camadas
- 1 O que é JWT e como funciona?

CAMADA DE DOMÍNIO

- 2 Entidade Pessoa — classe base com herança
- 3 Entidade Aluno — herda de Pessoa
- 4 Entidade Usuario — credenciais de acesso
- 5 Interface IAlunoRepository
- 6 Interface IUsuarioRepository
- 7 Interface ITokenService

CAMADA DE DADOS

- 8 ApplicationDbContext — mapeamento das entidades
- 9 AlunoRepository — implementação concreta
- 10 UsuarioRepository — consulta por email e senha

CAMADA DE APLICAÇÃO

- 11 Use Case: CriarAluno
- 12 Use Case: CriarUsuario
- 13 Use Case: RealizarLogin
- 14 TokenService — geração do JWT

CAMADA DE API

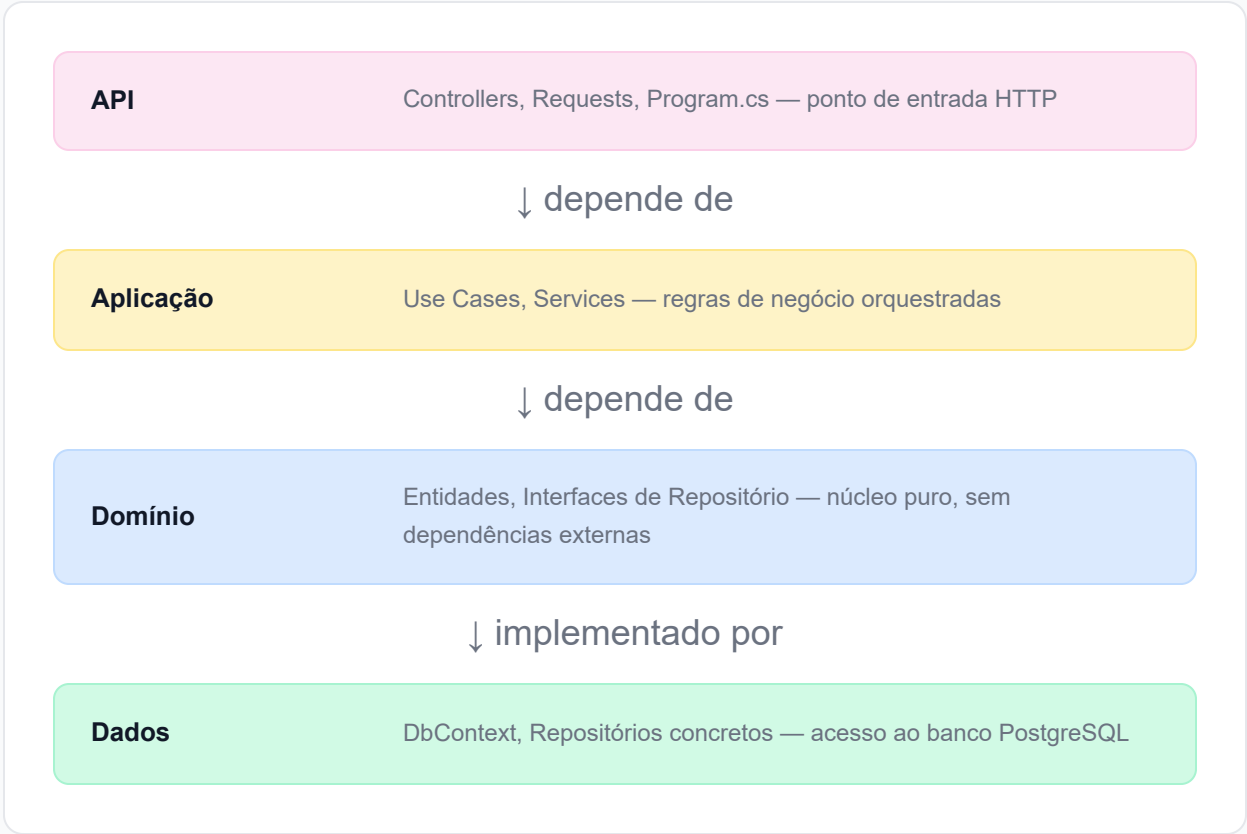
- 15 Requests — DTOs de entrada

- 16 AlunosController — endpoint protegido
- 17 UsuariosController — registro e login
- 18 Program.cs — configuração JWT + DI + Swagger

0

Arquitetura do Projeto — As 4 Camadas

O projeto usa **Clean Architecture** (Arquitetura Limpa) dividida em quatro projetos separados. Cada camada só pode depender das camadas internas — nunca ao contrário.



Por que separar? Se amanhã trocarmos PostgreSQL por SQL Server, só a camada *Dados* muda. A lógica de negócio (Aplicação) e as entidades (Domínio) ficam intactas.

Projeto (.csproj)	Responsabilidade	Pode referenciar
EinsteinGestaoAcademica.Dominio	Entidades e contratos	Nada (núcleo)
EinsteinGestaoAcademica.Dados	EF Core, SQL, repositórios	Domínio
EinsteinGestaoAcademica.Aplicacao	Use Cases, TokenService	Domínio
EinsteinGestaoAcademica.API	Controllers, DI, HTTP	Domínio + Aplicação + Dados

1

O que é JWT e como funciona?

JWT (*JSON Web Token*) é um padrão aberto (RFC 7519) para transmitir informações entre partes de forma compacta e segura. É amplamente usado para autenticação em APIs REST.

Estrutura de um Token JWT

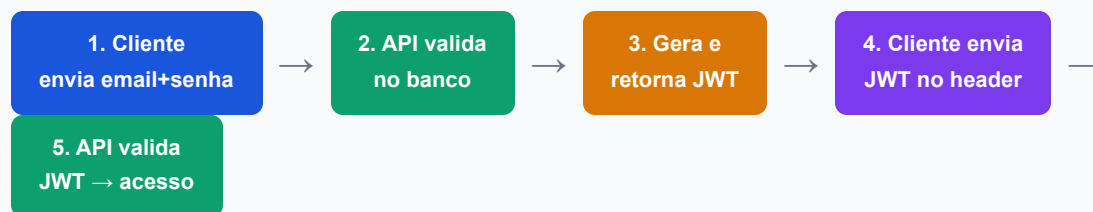
Um JWT é formado por **3 partes separadas por pontos**, cada uma codificada em Base64:

```
Exemplo de Token JWT

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9  ← Header (algoritmo + tipo)
.eyJzdWIiOiIxMjMlLCJlbWVpYCI6InRlc3RlQGVtYWlsLmNvbSI6ImV4cCI6MTcxOTAwMDAwMH0  ← Payload (claims)
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c  ← Signature (HMAC-SHA256 da chave secreta)
```

Parte	Conteúdo decodificado	Propósito
Header	<code>{"alg": "HS256", "typ": "JWT"}</code>	Identifica o algoritmo de assinatura
Payload	<code>{"sub": "1", "email": "x@y.com", "exp": ...}</code>	Dados do usuário (claims)
Signature	HMAC-SHA256(header+payload, chave_secreta)	Garante que o token não foi adulterado

Fluxo de Autenticação



No nosso projeto: O cliente chama `POST /api/usuarios/login` com email e senha. A API consulta o banco, gera um JWT com 8 horas de validade e retorna. Nas chamadas seguintes, o cliente envia o token no header `Authorization: Bearer <token>`. O `[Authorize]` nos controllers valida automaticamente.

2

Entidade Pessoa — Classe Base

DOMÍNIO

Por que criar Pessoa? Tanto Aluno quanto Professor são pessoas — têm nome, CPF, telefone, cidade, estado. Em vez de duplicar esses campos, criamos uma classe base comum. Isso é o princípio DRY (*Don't Repeat Yourself*) com herança.

EinsteinGestaoAcademica.Dominio / Entidades / Pessoa.cs

C#

```
namespace EinsteinGestaoAcademica.Dominio.Entidades
{
    public class Pessoa
    {
        public int id { get; set; }
        public string nome { get; set; }
        public string cpf { get; set; }
        public string telefone { get; set; }
        public string cidade { get; set; }
        public string estado { get; set; }
    }
}
```

Pontos de Atenção

Campo	Tipo	Obs.
id	int	Chave primária — gerada pelo banco
nome, cpf, telefone	string	Campos comuns a qualquer tipo de pessoa
cidade, estado	string	Endereço simplificado

3

Entidade Aluno — Herança de Pessoa

DOMÍNIO

Herança em EF Core: Aluno estende Pessoa com o campo `id_curso`, que é a chave estrangeira para o curso em que está matriculado. O campo `curso` é marcado com `[NotMapped]` — ele existe só em memória para facilitar o acesso ao objeto `Curso` quando necessário, mas não é salvo diretamente na tabela *aluno*.

```
EinsteinGestaoAcademica.Dominio / Entidades / Aluno.cs C#

using System.ComponentModel.DataAnnotations.Schema;

namespace EinsteinGestaoAcademica.Dominio.Entidades
{
    public class Aluno : Pessoa // ← herda todos os campos de Pessoa
    {
        public int id_curso { get; set; } // chave estrangeira

        [NotMapped] // EF Core não cria coluna para isto
        public Curso curso { get; set; }
    }
}
```

[NotMapped] diz ao Entity Framework para ignorar esse campo na hora de gerar SQL. É útil para propriedades calculadas ou de navegação que você não quer persistir no banco.

Tabela no Banco (TPT — Table Per Type)

No PostgreSQL temos duas tabelas: `pessoa` com os campos base e `aluno` com `id` (FK para `pessoa`) e `id_curso`.

4

Entidade Usuario — Credenciais de Acesso

DOMÍNIO

Por que Usuario é separado de Pessoa? Nem toda pessoa do sistema tem login (ex.: um aluno cadastrado pode ainda não ter uma conta). Separar as entidades dá mais flexibilidade: um Usuario pode estar associado a um aluno (id_aluno) ou a um professor (id_professor).

EinsteinGestaoAcademica.Dominio / Entidades / Usuario.cs

C#

```
namespace EinsteinGestaoAcademica.Dominio.Entidades
{
    public class Usuario
    {
        public int id { get; set; }
        public int? id_professor { get; set; } // nullable – pode não ter professor
        public int? id_aluno { get; set; }     // nullable – pode não ter aluno
        public string email { get; set; }
        public string senha { get; set; }
    }
}
```

Atenção — Segurança: Nesta implementação didática, a senha é armazenada em texto puro. Em produção, sempre use **hash + salt** (ex.: BCrypt ou PasswordHasher do ASP.NET Core Identity) para nunca armazenar a senha real no banco.

5

Interface IAlunoRepository

DOMÍNIO

Padrão Repository: A interface fica no Domínio — ela define o *que* pode ser feito com alunos no banco, sem saber *como*. A implementação concreta (que usa EF Core) fica na camada de Dados. Isso é o **Princípio de Inversão de Dependência** (o D do SOLID).

EinsteinGestaoAcademica.Dominio / Repositorios / IAlunoRepository.cs

C#

```
using EinsteinGestaoAcademica.Dominio.Entidades;

namespace EinsteinGestaoAcademica.Dominio.Repositorios
{
    public interface IAlunoRepository
    {
        Task Criar(Aluno aluno);
    }
}
```

6

Interface IUserRepository

DOMÍNIO

Dois métodos: Criar para registrar novo usuário, e ObterUsuarioPorEmailESenha para o processo de login — busca o usuário que bate com as credenciais fornecidas.

```
EinsteinGestaoAcademica.Dominio / Repositorios / IUserRepository.cs C#

using EinsteinGestaoAcademica.Dominio.Entidades;

namespace EinsteinGestaoAcademica.Dominio.Repositorios
{
    public interface IUserRepository
    {
        Task Criar(Usuario usuario);

        Task<Usuario> ObterUsuarioPorEmailESenha(string email, string senha);
    }
}
```

7

Interface ITokenService

DOMÍNIO

Por que a interface fica no Domínio? Manter `ITokenService` no Domínio permite que os Use Cases dependam apenas do contrato, não de uma biblioteca JWT específica. Se amanhã mudarmos de JWT para outro mecanismo, só a implementação concreta muda.

EinsteinGestaoAcademica.Dominio / ITokenService.cs

C#

```
using EinsteinGestaoAcademica.Dominio.Entidades;

namespace EinsteinGestaoAcademica.Dominio
{
    public interface ITokenService
    {
        string GerarToken(Usuario usuario);
    }
}
```

8

ApplicationDbContext — Mapeamento das Entidades

DADOS

DbContext é a porta de entrada do Entity Framework Core para o banco de dados. Cada `DbSet<T>` representa uma tabela. O método `OnModelCreating` configura como as classes C# se mapeiam para tabelas do PostgreSQL.

```
EinsteinGestaoAcademica.Dados / Data / ApplicationDbContext.cs C#

public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    public DbSet<Curso> Cursos { get; set; }
    public DbSet<Pessoa> Pessoas { get; set; }
    public DbSet<Aluno> Alunos { get; set; }
    public DbSet<Usuario> Usuarios { get; set; } // ← adicionado

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // ... mapeamento de Curso e Pessoa ...

        modelBuilder.Entity<Aluno>(e =>
        {
            e.ToTable("aluno", "public");
            e.Ignore(x => x.curso); // [NotMapped] alternativo
        });

        modelBuilder.Entity<Usuario>(e => // ← configuração nova
        {
            e.ToTable("usuario", "public");
            e.HasKey(x => x.id);
        });
    }
}
```

ToTable("usuario", "public") informa ao EF Core que a classe `Usuario` corresponde à tabela `public.usuario` no PostgreSQL (schema "public" é o padrão do Postgres).

9

AlunoRepository — Implementação Concreta

DADOS

Implementa **IALunoRepository** do Domínio usando o EF Core. Recebe o `ApplicationDbContext` por injeção de dependência — nunca instancia manualmente.

```
EinsteinGestaoAcademica.Dados / Data / Repositorios / AlunoRepository.cs C#

public class AlunoRepository : IAlunoRepository
{
    private readonly ApplicationDbContext _context;

    public AlunoRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task Criar(Aluno aluno)
    {
        await _context.Alunos.AddAsync(aluno); // adiciona ao rastreamento
        await _context.SaveChangesAsync();      // persiste no banco (INSERT)
    }
}
```

10

UsuarioRepository — Consulta por Email e Senha

DADOS

FromSqlRaw: Para a consulta de login, usamos SQL bruto parametrizado. Os parâmetros {0} e {1} são passados com segurança pelo EF Core — ele usa *prepared statements* internamente, protegendo contra SQL Injection.

```
EinsteinGestaoAcademica.Dados / Data / Repositorios / UsuarioRepository.cs C#

public class UsuarioRepository : IUsuarioRepository
{
    private readonly ApplicationDbContext _context;

    public UsuarioRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task Criar(Usuario usuario)
    {
        await _context.Usuarios.AddAsync(usuario);
        await _context.SaveChangesAsync();
    }

    public async Task<Usuario> ObterUsuarioPorEmailESenha(string email, string senha)
    {
        return await _context.Usuarios
            .FromSqlRaw("SELECT * FROM public.usuario WHERE email = {0} AND senha = {1}",
                email, senha)
            .FirstOrDefaultAsync(); // retorna null se não encontrar
    }
}
```

Nota de Segurança: FromSqlRaw com {0}, {1} usa parâmetros seguros (não interpolação direta). Nunca faça \$"...WHERE email = '{email}'" — isso abre vulnerabilidade de SQL Injection.

11

Use Case: CriarAluno

APLICAÇÃO

Use Case (ou Caso de Uso) representa uma operação específica do sistema. Ele orquestra o fluxo: recebe dados, aplica regras de negócio e chama o repositório. Cada Use Case tem **uma única responsabilidade**.

Interface (contrato)

Aplicacao / Services / Alunos / CriarAluno / ICriarAlunoUseCase.cs

C#

```
public interface ICriarAlunoUseCase
{
    Task CriarAluno(Aluno aluno);
}
```

Implementação

Aplicacao / Services / Alunos / CriarAluno / CriarAlunoUseCase.cs

C#

```
public class CriarAlunoUseCase : ICriarAlunoUseCase
{
    private readonly IAlunoRepository alunoRepository;

    public CriarAlunoUseCase(IAlunoRepository alunoRepository)
    {
        this.alunoRepository = alunoRepository;
    }

    public async Task CriarAluno(Aluno aluno)
    {
        await alunoRepository.Criar(aluno);
    }
}
```

Dependência invertida: O Use Case depende de IAlunoRepository (interface do Domínio), não de AlunoRepository (implementação da camada de Dados). Isso é o princípio D do SOLID em ação.

12

Use Case: CriarUsuario

APLICAÇÃO

Interface

Aplicacao / Services / Usuarios / CriarUsuario / ICriarUsuarioUseCase.cs

C#

```
public interface ICriarUsuarioUseCase
{
    Task CriarUsuario(Usuario usuario);
}
```

Implementação

Aplicacao / Services / Usuarios / CriarUsuario / CriarUsuarioUseCase.cs

C#

```
public class CriarUsuarioUseCase : ICriarUsuarioUseCase
{
    private readonly IUserRepository usuarioRepository;

    public CriarUsuarioUseCase(IUserRepository usuarioRepository)
    {
        this.usuarioRepository = usuarioRepository;
    }

    public async Task CriarUsuario(Usuario usuario)
    {
        await usuarioRepository.Criar(usuario);
    }
}
```

13

Use Case: RealizarLogin

APLICAÇÃO

Este Use Case é o gatilho da autenticação. Recebe email e senha, delega para o repositório a busca no banco e retorna o Usuario encontrado (ou null se não existir). O Controller é responsável por

verificar o null e gerar o token JWT.

Interface

Aplicacao / Services / Usuarios / RealizarLogin / IRealizarLoginUseCase.cs

C#

```
public interface IRealizarLoginUseCase
{
    Task<Usuario> RealizarLogin(string email, string senha);
}
```

Implementação

Aplicacao / Services / Usuarios / RealizarLogin / RealizarLoginUseCase.cs

C#

```
public class RealizarLoginUseCase : IRealizarLoginUseCase
{
    private readonly IUserarioRepository _usuarioRepository;

    public RealizarLoginUseCase(IUserarioRepository usuarioRepository)
    {
        _usuarioRepository = usuarioRepository;
    }

    public async Task<Usuario> RealizarLogin(string email, string senha)
    {
        return await _usuarioRepository.ObterUsuarioPorEmailESenha(email, senha);
    }
}
```

14

TokenService — Geração do JWT

APLICAÇÃO

Este é o coração da autenticação JWT. Recebe um `Usuario` autenticado e constrói um token assinado com chave secreta. O token contém *claims* (informações sobre o usuário) e tem validade de 8 horas.

```
EinsteinGestaoAcademica.Aplicacao / Services / Token / TokenService.cs C#

public class TokenService : ITokenService
{
    private readonly IConfiguration _config;

    public TokenService(IConfiguration config)
    {
        _config = config;
    }

    public string GerarToken(Usuario usuario)
    {
        // 1. Claims: dados que ficam dentro do token
        var claims = new[]
        {
            new Claim(ClaimTypes.NameIdentifier, usuario.id.ToString()),
            new Claim(ClaimTypes.Email, usuario.email),
        };

        // 2. Chave de assinatura (vem do appsettings.json)
        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(_config["Jwt:Key"]));

        // 3. Credenciais de assinatura (algoritmo HMAC-SHA256)
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

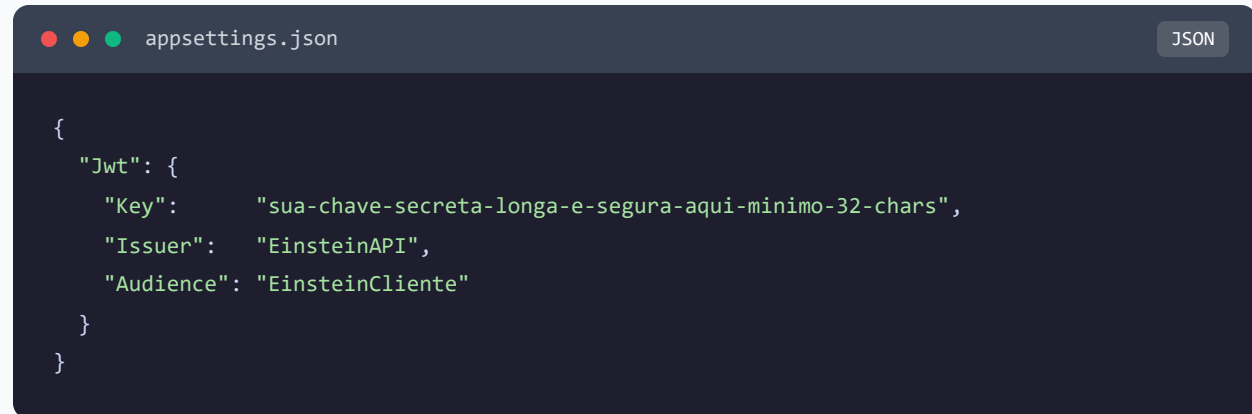
        // 4. Constrói o token JWT
        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"], // quem emitiu
            audience: _config["Jwt:Audience"], // para quem é
            claims: claims,
            expires: DateTime.Now.AddHours(8), // validade
            signingCredentials: creds
        );

        // 5. Serializa para string (o JWT que o cliente recebe)
        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

```
}  
}
```

Configuração no appsettings.json

As chaves lidas via `_config["Jwt:..."]` vêm do arquivo `appsettings.json`:



```
appsettings.json  
JSON  
{  
  "Jwt": {  
    "Key": "sua-chave-secreta-longa-e-segura-aqui-minimo-32-chars",  
    "Issuer": "EinsteinAPI",  
    "Audience": "EinsteinCliente"  
  }  
}
```

Jwt:Key deve ter pelo menos 32 caracteres para HMAC-SHA256. Em produção, nunca coloque a chave no `appsettings.json` versionado — use variáveis de ambiente ou Azure Key Vault.

15

Requests — DTOs de Entrada

API

Request ≠ Entidade. Os objetos Request (DTO — Data Transfer Object) representam exatamente o que o cliente deve enviar no corpo da requisição HTTP. Eles são desacoplados das entidades do Domínio — o Controller faz a conversão entre os dois.

CriarAlunoRequest

API / Requests / CriarAlunoRequest.cs

C#

```
public class CriarAlunoRequest
{
    public string nome { get; set; }
    public string cpf { get; set; }
    public string telefone { get; set; }
    public string cidade { get; set; }
    public string estado { get; set; }
    public int id_curso { get; set; }
}
```

CriarUsuarioRequest

API / Requests / CriarUsuarioRequest.cs

C#

```
public class CriarUsuarioRequest
{
    public int? id_professor { get; set; }
    public int? id_aluno { get; set; }
    public string email { get; set; }
    public string senha { get; set; }
}
```

RealizarLoginRequest

API / Requests / RealizarLoginRequest.cs

C#

```
public class RealizarLoginRequest
{
    public string email { get; set; }
}
```

```
public string senha { get; set; }  
}
```

16

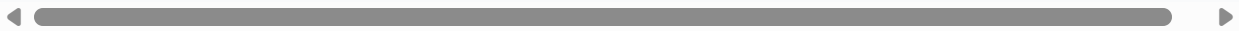
AlunosController — Endpoint Protegido

API

[Authorize] na classe inteira significa que *todos* os endpoints deste controller exigem um JWT válido no header. Se o token estiver ausente, expirado ou inválido, o ASP.NET Core retorna automaticamente **401 Unauthorized**.

```
EinsteinGestaoAcademica.API / Controllers / AlunosController.cs C#  
  
[ApiController]  
[Authorize] // ← todos os endpoints exigem JWT válido  
[Route("api/[controller]")] // ← rota: /api/alunos  
public class AlunosController : ControllerBase  
{  
    private readonly ICriarAlunoUseCase criarAlunoUseCase;  
  
    public AlunosController(ICriarAlunoUseCase criarAlunoUseCase)  
    {  
        this.criarAlunoUseCase = criarAlunoUseCase;  
    }  
  
    [HttpPost] // POST /api/alunos  
    public async Task<IActionResult> CriarAluno([FromBody] CriarAlunoRequest criarAlunoRequest)  
    {  
        try  
        {  
            await criarAlunoUseCase.CriarAluno(new Aluno()  
            {  
                nome = criarAlunoRequest.nome,  
                cpf = criarAlunoRequest.cpf,  
                estado = criarAlunoRequest.estado,  
                cidade = criarAlunoRequest.cidade,  
                telefone = criarAlunoRequest.telefone,  
                id_curso = criarAlunoRequest.id_curso  
            });  
            return Created(); // HTTP 201  
        }  
        catch (Exception)
```

```
    {  
        return StatusCode(500);  
    }  
}  
}
```



17

UsuariosController — Registro e Login

API

Combinação de [Authorize] e [AllowAnonymous]: O controller inteiro é protegido com [Authorize], mas o endpoint de login precisa ser público — por isso tem [AllowAnonymous] individualmente. Este é o padrão correto: proteger tudo por padrão e liberar seletivamente.

```
EinsteinGestaoAcademica.API / Controllers / UsuariosController.cs C#

[ApiController]
[Authorize]
[Route("api/[controller]")]
public class UsuariosController : ControllerBase
{
    private readonly ICriarUsuarioUseCase criarUsuarioUseCase;
    private readonly IRealizarLoginUseCase realizarLoginUseCase;
    private readonly ITokenService tokenService;

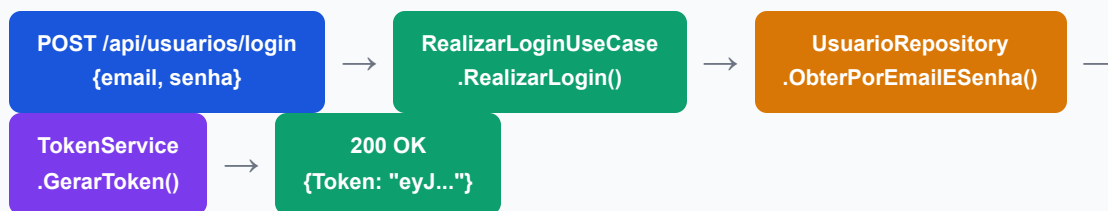
    public UsuariosController(ICriarUsuarioUseCase criarUsuarioUseCase,
                             IRealizarLoginUseCase realizarLoginUseCase,
                             ITokenService tokenService)
    {
        this.criarUsuarioUseCase = criarUsuarioUseCase;
        this.realizarLoginUseCase = realizarLoginUseCase;
        this.tokenService = tokenService;
    }

    [HttpPost] // POST /api/usuarios - requer JWT
    public async Task<IActionResult> CriarUsuario([FromBody] CriarUsuarioRequest request)
    {
        try
        {
            await criarUsuarioUseCase.CriarUsuario(new Usuario()
            {
                id_aluno = request.id_aluno,
                id_professor = request.id_professor,
                email = request.email,
                senha = request.senha
            });
            return Created();
        }
        catch (Exception) { return StatusCode(500); }
    }
}
```

```
[HttpPost("login")] // POST /api/usuarios/login – público
[AllowAnonymous] // ← libera este endpoint para não-autenticados
public async Task<IActionResult> RealizarLogin([FromBody] RealizarLoginRequest request)
{
    try
    {
        var usuario = await realizarLoginUseCase.RealizarLogin(request.email, request.senha);
        if (usuario == null)
            return Unauthorized(); // HTTP 401 – credenciais inválidas

        var token = tokenService.GerarToken(usuario);
        return Ok(new { Token = token }); // HTTP 200 com o JWT
    }
    catch (Exception) { return StatusCode(500); }
}
```

Fluxo do Login



18

Program.cs — Configuração JWT + DI + Swagger

API

Program.cs é a cola da aplicação. Aqui registramos todos os serviços no container de Injeção de Dependência, configuramos o JWT e ajustamos o Swagger para suportar o botão "Authorize" com Bearer Token.

Parte 1 — Configuração JWT Bearer

```
Program.cs - AddAuthentication + AddJwtBearer C#

// Pacote necessário na camada Aplicacao:
// Microsoft.AspNetCore.Authentication.JwtBearer 8.0.x

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer          = true,    // valida o campo "iss"
            ValidateAudience       = true,    // valida o campo "aud"
            ValidateLifetime        = true,    // valida expiração
            ValidateIssuerSigningKey = true,    // valida a assinatura

            ValidIssuer      = configuration["Jwt:Issuer"],
            ValidAudience   = configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(configuration["Jwt:Key"]))
        };
    });
```

Parte 2 — Swagger com suporte a Bearer Token

```
Program.cs - AddSwaggerGen com SecurityDefinition C#

builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo { Title = "Einstein API", Version = "v1" });

    // Botão "Authorize" no Swagger UI
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name      = "Authorization",
```

```
        Type = SecuritySchemeType.Http,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Insira o token JWT"
    });
    options.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme {
                Reference = new OpenApiReference {
                    Type = ReferenceType.SecurityScheme, Id = "Bearer"
                },
                Array.Empty<string>()
            }
        }
    });
});
```

Parte 3 — Registro de Dependências (DI)

```
Program.cs — AddTransient (todos os serviços) C#

// Cursos
builder.Services.AddTransient<ICriarCursoUseCase, CriarCursoUseCase>();
builder.Services.AddTransient<IAltararCursoUseCase, AlterarCursoUseCase>();
builder.Services.AddTransient<IRemoverCursoUseCase, RemoverCursoUseCase>();
builder.Services.AddTransient<ICursoRepositorio, CursoRepositorio>();

// Alunos
builder.Services.AddTransient<ICriarAlunoUseCase, CriarAlunoUseCase>();
builder.Services.AddTransient<IALunoRepositorio, AlunoRepositorio>();

// Usuários
builder.Services.AddTransient<ICriarUsuarioUseCase, CriarUsuarioUseCase>();
builder.Services.AddTransient<IRealizarLoginUseCase, RealizarLoginUseCase>();
builder.Services.AddTransient<IUsuarioRepositorio, UsuarioRepositorio>();

// Token JWT
builder.Services.AddTransient<ITokenService, TokenService>();
```

Parte 4 — Pipeline HTTP (ordem importa!)

```
Program.cs — middleware pipeline C#

app.UseSwagger();
app.UseSwaggerUI(...);
```

```
app.UseAuthentication(); // ← DEVE vir ANTES de UseAuthorization
app.UseAuthorization(); // ← verifica os [Authorize]

app.MapControllers();
app.Run();
```

Ordem do middleware é crítica! `UseAuthentication()` deve sempre vir antes de `UseAuthorization()`. A autenticação identifica *quem* é o usuário; a autorização decide *o que* ele pode fazer. Inverter a ordem faz os `[Authorize]` nunca funcionarem.

Resumo Final — Fluxo Completo

Etapa	Endpoint	Requer JWT?	Resposta
1. Criar Usuário	POST /api/usuarios	Sim	201 Created
2. Login	POST /api/usuarios/login	Não	200 OK + {Token}
3. Criar Aluno	POST /api/alunos	Sim	201 Created
4. Sem token	Qualquer endpoint com [Authorize]	—	401 Unauthorized
5. Token inválido	Qualquer endpoint com [Authorize]	—	401 Unauthorized